

Laboratorul 4

Procese

1 Ce este un proces?

Un sistem de operare execută o varietate de *programe* care rulează ca și *proces*.

Program	Proces
Un fișier executabil static, cu instrucțiuni și date, stocat pe disk, care nu face nimic până nu este lansat în execuție.	Un program aflat în execuție, cu o stare proprie în memorie, resurse alocate și un context de execuție activ.

→ Un program devine proces atunci când un fișier executabil este încărcat în memorie.

Analogie: *programul* este rețeta, *procesul* este gătitul → același program poate genera mai multe procese simultan, fiecare independent de celelalte.

Informațiile despre procese sunt ținute într-o structură numită **Process Control Block (PCB)**, numită și **task control block**, câte una pentru fiecare process existent în sistem. Printre cele mai importante informații conținute de PCB regăsim:

- PID (process ID);
- PPID (parent process ID);
- spațiul de adrese;
- starea procesului – running, waiting, etc;
- registre generale, PC (contor program), SP (indicator stivă);
- tabela de fișiere deschise;
- informații referitoare la semnale;
- informații referitoare la sistemele de fișiere (directorul rădăcină).

process state
process number
program counter
registers
CPU schedule info
memory limits
list of open files
...

Spațiul de adrese al unui proces este izolat de cel al celorlalte procese și se împarte în mai multe zone:

- **Zona de cod** (*text section*) – instrucțiunile programului;
- **Zona de date** (*data section*) – variabilele globale și statice;
- **Heap-ul** – memorie alocată dinamic (ex. malloc);

- **Stiva** (*stack*) – date temporare precum: parametrii funcțiilor, adrese de return, variabile locale.

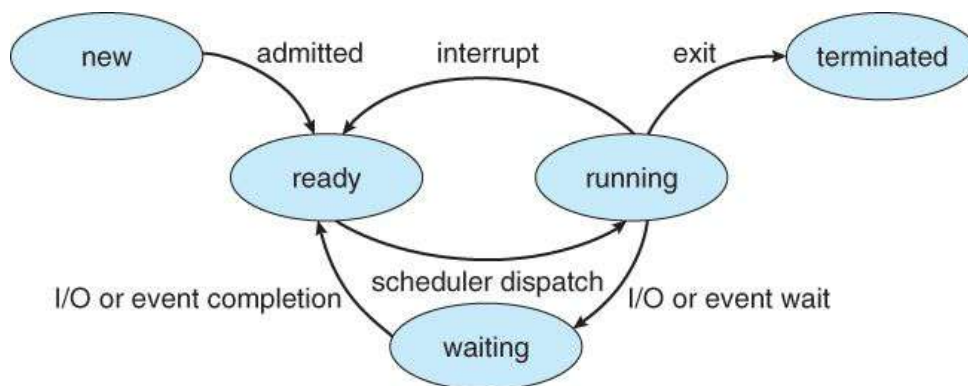


Figura 1: Diagrama stărilor unui proces.

Sursă: Abraham Silberschatz, Peter B. Galvin, Greg Gagne, “Operating System Concepts”, 10th Edition.

Stările unui proces

- **New:** procesul este inițiat;
- **Running:** procesul rulează;
- **Waiting:** procesul așteaptă un eveniment;
- **Ready:** procesul așteaptă să fie alocat unui procesor;
- **Terminated:** procesul și-a terminat execuția.

Starea zombie. Un proces terminat care nu a fost încă *recoltat* de părintele său (prin `wait(2)`) rămâne în sistem ca proces *zombie* – nu mai execută nimic, dar ocupă o intrare în tabela de procese până când părintele îi citește codul de retur. De aceea este important ca orice proces părinte să apeleze `wait(2)` după ce creează copii.

Scheduler-ul este componenta din cadrul *kernelului* care se ocupă de gestionarea proceselor. Acesta selectează procesele și le atribuie unuia dintre *core-urile* procesorului pentru a fi rulate în funcție de prioritate, bazându-se pe o serie de algoritmi.

Sistemul de fișiere `/proc`

Linux expune informații despre procesele active printr-un sistem de fișiere virtual numit `/proc`. Acesta nu există pe disk, ci este generat de kernel în timp real și conține câte un director pentru fiecare proces activ, denumit după PID-ul său.

```
$ ls /proc/
1/      1234/    123456/  cpuinfo  meminfo  sys/    tty/    ...
```

În interiorul directorului unui proces se găsesc fișiere care expun informațiile din task control block-ul acestuia:

- `cmdline` – comanda cu care a fost lansat procesul;
- `status` – stare, PID, PPID, utilizator, memorie utilizată;
- `fd/` – directorul cu fișierele deschise ale procesului;

- `maps` – harta spațiului de adrese (cod, date, heap, stivă);
- `cwd` – symlink către directorul curent al procesului.

Puteți inspecta procesul curent al shell-ului folosind variabila specială `$$`, care conține PID-ul shell-ului activ:

```
$ ls /proc/$$
$ cat /proc/$$/status
$ ls -la /proc/$$/fd
```

Bonus: Nu toate sistemele de operare pot rula mai multe procese simultan. Sistemele **monotasking** (ex. MS-DOS) rulează doar un singur proces la un moment dat. Sistemele **multitasking** (ex. UNIX, Linux, Windows) pot rula mai multe procese simultan, pe procesoare diferite sau alternat pe același procesor, creând iluzia execuției paralele.

2 Crearea unui proces nou

În mediile de dezvoltare UNIX, funcția sistem cu care se creează procese noi este `fork(2)`. Odată invocată, funcția creează un proces nou (numit *proces copil*), acesta fiind o copie a procesului apelant, dar cu câteva diferențe, dintre care le enumerăm aici pe cele mai importante (ele pot diferi de la sistem de operare la sistem de operare):

- procesul copil are ca părinte chiar pe procesul ce a apelat `fork(2)` (în vreme ce procesul apelant va avea un alt părinte);
- procesul copil are un ID (denumit și `pid`) diferit de cel al părintelui;
- procesul copil are un singur fir de execuție (*thread*, v. Laboratorul 6);
- procesul copil pornește de la zero în ceea ce privește resursele utilizate și timpul de execuție, precum și alți indicatori similari de gestionare a proceselor.

Din momentul apelului, dacă acesta este încheiat cu succes, fiecare viitoare instrucțiune va fi executată atât de părinte, cât și de copil. Diferențierea se face în funcție de valoarea de retur a lui `fork(2)`: copilul primește valoarea 0 iar părintele `pid`-ul copilului. Astfel, o construcție tipică C este:

```
pid_t pid = fork();
if (pid < 0)
    return errno;
else if (pid == 0)
    /* child instructions */
else
    /* parent instructions */
```

Oricând în timpul execuției putem afla `pid`-ul procesului curent și pe cel al procesului părinte cu ajutorul funcțiilor `getpid(2)` și, respectiv, `getppid(2)`.

```
printf("Parent %d Me %d\n", getppid(), getpid());
```

Părintele își poate suspenda activitatea pentru a aștepta finalizarea execuției unui proces copil cu ajutorul funcției `wait(2)`, care oferă ca valoare de retur pid-ul copilului.

Operația este utilă pentru a sincroniza și ordona instrucțiunile:

```
pid_t pid = fork();
if (pid < 0)
    return errno;
else if (pid == 0) {
    /* child instructions */
    printf("First!");
} else {
    /* parent instructions */
    wait(NULL);
    printf("Last!");
}
```

Atenție: Funcția `wait(2)` redă control părintelui când se termină execuția *oricăruia* dintre copiii săi. Pentru cazuri complexe, în care se dorește așteptarea unuia sau mai multor procese anume, se pot folosi funcții avansate precum `waitpid(2)` sau `wait4(2)`, dar care nu fac obiectul laboratorului.

3 Executarea unui program existent

Executarea unui program se realizează cu ajutorul apelului sistem `execve(2)`.

```
int execve(const char *path, char *const argv[], char *const envp
[]);
```

Aceasta suprascrie complet procesul apelant cu un nou proces, conform programului găsit la calea indicată în `path`.

Atenție: Calea trebuie să fie absolută, de exemplu `/bin/pwd` și nu `pwd`. Pentru a obține această cale, puteți folosi comanda `which(1)`:

```
$ which pwd
/bin/pwd
$ which vi
/usr/bin/vi
```

Argumentele programului sunt puse în `argv` respectând convenția obișnuită din C: pe prima poziție (`argv[0]`) se află calea absolută către program urmată de argumente. Lista se încheie cu `null`. Variabilele de sistem din mediul de execuție sunt puse în ultimul argument `envp`. Aceasta este o listă de șiruri de caractere similară cu `argv` exceptând convenția primului element.

Din cauza efectului distructiv asupra procesului curent, `execve(2)` este adesea folosit împreună cu `fork(2)` astfel încât procesul nou-creat să fie cel suprascris.

```

pid_t pid = fork();
if (pid < 0)
    return errno;
else if (pid == 0) {
    /* child instructions */
    char *argv[] = {"pwd", NULL};
    execve("/bin/pwd", argv, NULL);
    perror(NULL);
} else
    /* parent instructions */

```

Având în vedere suprascrierea procesului curent, `execve(2)` nu mai revine în programul inițial decât în cazul în care a apărut o eroare, folosindu-se `errno` pentru a determina cauza. Cele mai des întâlnite erori sunt calea greșită sau lipsa lui `argv`.

4 Sarcini de laborator

1. Creați un proces nou folosind `fork(2)` și afișați fișierele din directorul curent cu ajutorul `execve(2)`. Din procesul inițial afișați pid-ul propriu și pid-ul copilului. De exemplu:

```

$ ./forkls
My PID=41875, Child PID=62668
Makefile collatz.c forkl.c so-lab-4.tex
collatz forkl ncollatz.c
Child 62668 finished

```

2. Ipoteza Collatz spune că plecând de la orice număr natural, dacă aplicăm repetat următoarea operație:

$$n \mapsto \begin{cases} n/2 & \text{dacă } n \equiv 0 \pmod{2} \\ 3n + 1 & \text{dacă } n \not\equiv 0 \pmod{2} \end{cases}$$

șirul ce rezultă va atinge valoarea 1. Implementați un program care folosește `fork(2)` și testează ipoteza generând șirul asociat unui număr dat în procesul copil.

Exemplu:

```

$ ./collatz 24
24: 24 12 6 3 10 5 16 8 4 2 1.
Child 52923 finished

```

3. Implementați un program care să testeze ipoteza Collatz pentru mai multe numere date. Pornind de la un singur proces părinte, este creat câte un copil care se ocupă de un singur număr. Părintele va aștepta să termine execuția fiecare copil. Programul va demonstra acest comportament folosind funcțiile `getpid(2)` și `getppid(2)`. Exemplu:

```
$ ./ncollatz 9 16 25 36
Starting parent 6202
9: 9 28 14 7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1.
36: 36 18 9 28 14 7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2
    1.
Done Parent 6202 Me 40018
Done Parent 6202 Me 30735
16: 16 8 4 2 1.
25: 25 76 38 19 58 29 88 44 22 11 34 17 52 26 13 40 20 10 5
    16 8 4 2 1.
Done Parent 6202 Me 13388
Done Parent 6202 Me 98514
Done Parent 58543 Me 6202
```